

```

// netlify/functions/ai-proxy.js
//
// Les 5 fonctions IA de Génération Capable (plan du jour, coaching, évaluation,
// rapport du soir, scan vendeur) appelaient https://api.anthropic.com directemen
// depuis le navigateur. Ce pattern ne fonctionne QUE dans l'aperçu Artifacts de
// claude.ai (qui proxifie la clé automatiquement) – une fois le site déployé sur
// Netlify, cet appel échoue à 100% (pas de clé envoyée, et de toute façon les
// navigateurs bloquent les appels directs à l'API Anthropic pour des raisons de
// sécurité : une clé visible côté client serait volable par n'importe qui).
//
// Cette fonction tourne côté serveur (jamais visible du navigateur).
// Le site a une clé OPENAI_API_KEY configurée dans Netlify (pas de clé Anthropic
// pour l'instant). La fonction gère les deux automatiquement :
// 1) si ANTHROPIC_API_KEY existe → utilise Claude
// 2) sinon, si OPENAI_API_KEY existe → utilise la clé OpenAI déjà en place
// 3) sinon → message "Analyse indisponible" propre (bouton Réessayer déjà en p
// Dans tous les cas, la réponse renvoyée au front-end a exactement la même forme
// (celle de l'API Anthropic : {content:[{type:'text', text:'...'}]}) donc index.
// n'a besoin d'aucun changement supplémentaire, quel que soit le fournisseur uti
//
// FIX (audit 2026-07-21) :
// 1) Toutes les erreurs (clé absente, erreur API, réseau) étaient renvoyées av
// statusCode 200 dans un champ {error:...}. Le front-end ne lisait jamais c
// champ : il cherchait directement data.content, qui était undefined, ce qu
// faisait planter le JSON.parse() côté front avec un message générique
// "Analyse indisponible" qui masquait la VRAIE cause. On garde le statusCod
// (pour compat front) mais on logge maintenant l'erreur réelle côté serveur
// (visible dans Netlify → Functions → ai-proxy → Logs).
// 2) max_tokens envoyé par le front (1000-1200) est trop juste pour les schéma
// demandés (5 actions détaillées, 3 axes d'amélioration, etc.) avec gpt-4o-
// Une réponse tronquée = JSON invalide = échec silencieux. On relève le pla
// à 2000 côté proxy et on force response_format:json_object côté OpenAI pou
// garantir un JSON valide (gpt-4o-mini supporte ce paramètre).
//
// FIX (GC Cognitive Engine – intégration system prompt) :
// 3) index.html envoie désormais un champ `system` (GC_SYSTEM_PROMPT) dans le
// payload, distinct de `messages`. Les deux fournisseurs ne le consomment p
// pareil :
// - Anthropic /v1/messages accepte un paramètre top-level `system` séparé
// des messages → on le transmet tel quel.
// - OpenAI /v1/chat/completions n'a pas de paramètre `system` dédié : le
// system prompt doit être le premier message avec role:'system'. On le
// construit ici, sans jamais dépendre de ce que le front envoie dans
// `messages` (qui ne contient qu'un message role:'user').

```

```

//      Si `system` est absent du payload (appel legacy), le comportement précède
//      est conservé à l'identique pour les deux fournisseurs.

exports.handler = async (event) => {
  if (event.httpMethod !== 'POST') {
    return { statusCode: 405, body: JSON.stringify({ error: 'Method not allowed'
    })
  }

  let payload;
  try {
    payload = JSON.parse(event.body || '{}');
  } catch (e) {
    console.error('[ai-proxy] Invalid JSON body from client:', e.message);
    return { statusCode: 400, body: JSON.stringify({ error: 'Invalid JSON body' })
  }

  const { messages, system } = payload;
  if (!messages) {
    return { statusCode: 400, body: JSON.stringify({ error: 'Missing messages' })
  }
  const max_tokens = Math.max(payload.max_tokens || 1000, 2000);

  const anthropicKey = process.env.ANTHROPIC_API_KEY;
  const openaiKey = process.env.OPENAI_API_KEY;

  console.log('[ai-proxy] Clés présentes -> ANTHROPIC:', !!anthropicKey, '| OPENAI', !!openaiKey);

  if (!anthropicKey && !openaiKey) {
    console.error('[ai-proxy] AUCUNE clé API configurée sur Netlify.');
```

```

    return ok({
      error: 'NO_API_KEY',
      message: "Aucune clé IA configurée sur Netlify (ANTHROPIC_API_KEY ou OPENAI_API_KEY)"
    });
  }

  try {
    if (anthropicKey) {
      const anthropicBody = {
        model: payload.model || 'claude-sonnet-4-6',
        max_tokens,
        messages
      };
      // Anthropic attend le system prompt comme paramètre top-level dédié,
      // jamais mélangé dans le tableau messages.
      if (system) anthropicBody.system = system;

      const r = await fetch('https://api.anthropic.com/v1/messages', {

```

```

    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'x-api-key': anthropicKey,
      'anthropic-version': '2023-06-01'
    },
    body: JSON.stringify(anthropicBody)
  });
const data = await r.json();
if (!r.ok) {
  console.error('[ai-proxy] Erreur Anthropic', r.status, JSON.stringify(data));
  return ok({ error: 'ANTHROPIC_API_ERROR', status: r.status, detail: data });
}
return ok(data); // déjà au bon format {content:[{type:'text',text:...}]}
}

// Pas de clé Anthropic pour l'instant : on utilise la clé OpenAI déjà config
// OpenAI n'a pas de paramètre `system` séparé : on le préfixe comme premier
// message role:'system'. On ne fait jamais confiance à ce que le front met d
// `messages` pour ce rôle – c'est reconstruit ici de façon déterministe.
const openaiMessages = system
  ? [{ role: 'system', content: system }, ...messages]
  : messages;

const r = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${openaiKey}`
  },
  body: JSON.stringify({
    model: 'gpt-4o-mini',
    max_tokens,
    response_format: { type: 'json_object' },
    messages: openaiMessages
  })
});
const data = await r.json();
if (!r.ok) {
  console.error('[ai-proxy] Erreur OpenAI', r.status, JSON.stringify(data));
  return ok({ error: 'OPENAI_API_ERROR', status: r.status, detail: data });
}
const text = data.choices?.[0]?.message?.content || '';
if (!text) {
  console.error('[ai-proxy] Réponse OpenAI vide ou tronquée:', JSON.stringify
}
// Normalisé à la forme Anthropic pour que index.html n'ait besoin d'aucun ch

```

```
    return ok({ content: [{ type: 'text', text }] });

  } catch (e) {
    console.error('[ai-proxy] NETWORK_ERROR', e);
    return ok({ error: 'NETWORK_ERROR', message: String(e) });
  }
};

function ok(data) {
  return { statusCode: 200, headers: { 'Content-Type': 'application/json' }, body
}
```